```
================================================================================
🚀 ТЕСТ ДЕТЕРМИНИРОВАННОГО GPU DROPOUT ВНУТРИ LOCALRUNNER
================================================================================
Выполнение прямого прохода JIT-ядра (Умножение + Dropout, seed=424242)...
C:\Users\Дмитрий\AppData\Local\Programs\Python\Python313\Lib\site-packages\pyopencl\
__init__.py:581: CompilerWarning: Non-empty compiler output encountered. Set the environment
variable PYOPENCL_COMPILER_OUTPUT=1 to see more.
  lambda: self._prg.build(options_bytes, devices),
Выполнение обратного прохода градиентов...

################################################################################
🔍 ПОШАГОВАЯ ТРАССИРОВКА ОБРАТНОГО ПРОХОДА (DEEP DETAILED LOG)
################################################################################

[ЗАДАЧА НАЗАД №1]: Вызов ядра fused_backward_445752684522486188
------------------------------------------------
Содержимое буферов во VRAM ПЕРЕД вычислением текущего шага:
  • X              -> [ 2.  -1.   3.   4.  -2.   1.5  0.5 -0.5]
  • grad_X         -> [0. 0. 0. 0. 0. 0. 0. 0.]
  • W              -> [1. 2. 1. 2. 1. 2. 1. 2.]
  • grad_W         -> [0. 0. 0. 0. 0. 0. 0. 0.]
  • Y              -> [ 2.6667  0.      4.      0.      0.      4.      0.6667 -1.3333]
  • grad_Y         -> [1. 1. 1. 1. 1. 1. 1. 1.]
4 errors generated.
Traceback (most recent call last):
  File "C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests\test_integration_fused.py",
line 189, in <module>
    test_integration_dropout_fused()
    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^
  File "C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests\test_integration_fused.py",
line 159, in test_integration_dropout_fused
    runner.run_backward()
    ~~~~~~~~~~~~~~~~~~~^^
  File "C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\compiler\nncl_compiler.py", line
289, in run_backward
    program = JITProgramCache.get_or_compile(self.ctx, code)
  File "C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\compiler\nncl_compiler.py", line
17, in get_or_compile
    cls._cache[kernel_code] = cl.Program(ctx, kernel_code).build()
                              ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^
  File "C:\Users\Дмитрий\AppData\Local\Programs\Python\Python313\Lib\site-packages\
pyopencl\__init__.py", line 580, in build
    self._build_and_catch_errors(
    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~^
        lambda: self._prg.build(options_bytes, devices),
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
        options_bytes=options_bytes)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Дмитрий\AppData\Local\Programs\Python\Python313\Lib\site-packages\
pyopencl\__init__.py", line 638, in _build_and_catch_errors
    raise err
pyopencl._cl.RuntimeError: clBuildProgram failed: BUILD_PROGRAM_FAILURE -
clBuildProgram failed: BUILD_PROGRAM_FAILURE - clBuildProgram failed:
BUILD_PROGRAM_FAILURE
```

Build on <pyopencl.Device 'NVIDIA GeForce RTX 2060' on 'NVIDIA CUDA' at 0x2119d745f90>:

```
<kernel>:182:11: error: redefinition of 'upstream_Y'
   float upstream_Y = grad_Y[tid];
         ^
<kernel>:178:23: note: previous definition is here
             float upstream_Y = grad_Y[tid];
                   ^
<kernel>:183:23: error: expected ';' at end of declaration
   float back_nnclloc.dropout(X = upstream_Y * W, 0.25f, tid, 424242)[tid];
                   ^
                   ;
<kernel>:184:19: error: expected identifier or '('
   float back_W, 0.25f, tid, 424242) = upstream_Y * nnclloc.dropout(X[tid];
              ^
<kernel>:184:18: error: expected ';' at end of declaration
   float back_W, 0.25f, tid, 424242) = upstream_Y * nnclloc.dropout(X[tid];
             ^
             ;
```

(options: -I C:\Users\Дмитрий\AppData\Local\Programs\Python\Python313\Lib\site-packages\pyopencl\cl)

```python
    def run_backward(self):
        if not self.backward_stack:
            print("-> Стек задач Backward пуст.")
            return

        print("\n" + "#"*70)
        print("🔍 ПОШАГОВАЯ ТРАССИРОВКА ОБРАТНОГО ПРОХОДА (DEEP DETAILED LOG)")
        print("#"*70)

        from compiler.host_returner import HostReturner
        import re

        task_idx = 1
        while self.backward_stack:
            code, name, buffers, total_elements = self.backward_stack.pop()

            print(f"\n[ЗАДАЧА НАЗАД №{task_idx}]: Вызов ядра {name}")
            print("-" * 50)

            # Извлекаем сигнатуру аргументов из сгенерированного Си-кода ядра
            # Нам нужно понять, в каком порядке идут указатели на буферы
            args_match = re.search(r'__kernel void \w+\((.*?)\)', code, re.DOTALL)
            if args_match:
                # Получаем список имён переменных из Си-кода, например: ['gate_A', 'grad_gate_A', 'saved_gate_A']
                arg_declarations = args_match.group(1).split(',')
                arg_names = [decl.strip().split()[-1].replace('*', '') for decl in arg_declarations if decl.strip()]
                # Последний аргумент обычно const int total_elements, отсекаем его
                if 'total_elements' in arg_names[-1]:
                    arg_names = arg_names[:-1]
            else:
                arg_names = [f"buffer_{i}" for i in range(len(buffers))]

            # Делаем дамп содержимого ВСЕХ задействованных буферов ДО выполнения текущего ядра
            print("Содержимое буферов во VRAM ПЕРЕД вычислением текущего шага:")
            with np.printoptions(precision=4, suppress=True, edgeitems=8, linewidth=80):
                for name_buf, cl_buf in zip(arg_names, buffers):
                    # Создаем временный суррогатный объект, чтобы HostReturner смог прочитать сырой cl.Buffer
                    class TargetWrapper:
                        def __init__(self, buf, size, queue):
                            self.data_buffer = buf
                            self.shape = (size,)
                            self.dtype = np.dtype(np.float32)
                            self.queue = queue

                    wrapper = TargetWrapper(cl_buf, total_elements, self.queue)
                    cpu_data = HostReturner.to_cpu(wrapper)
                    print(f"  • {name_buf:18} -> {cpu_data}")
```

```python
                            self.data_buffer = buf
                            self.shape = (size,)
                            self.dtype = np.dtype(np.float32)
                            self.queue = queue

                    wrapper = TargetWrapper(cl_buf, total_elements, self.queue)
                    cpu_data = HostReturner.to_cpu(wrapper)
                    print(f"  • {name_buf:18} -> {cpu_data}")

            # Компилируем и запускаем ядро на GPU
            program = JITProgramCache.get_or_compile(self.ctx, code)
            kernel = getattr(program, name)
            kernel(self.queue, (total_elements,), None, *buffers, np.int32(total_elements))
            self.queue.finish() # Синхронизируем, чтобы увидеть изменения

            # Делаем дамп результатов ПОСЛЕ выполнения ядра
            print("\nСодержимое буферов во VRAM ПОСЛЕ вычисления текущего шага:")
            with np.printoptions(precision=4, suppress=True, edgeitems=8, linewidth=80):
                for name_buf, cl_buf in zip(arg_names, buffers):
                    wrapper = TargetWrapper(cl_buf, total_elements, self.queue)
                    cpu_data = HostReturner.to_cpu(wrapper)
                    print(f"  • {name_buf:18} -> {cpu_data}")

            print("-" * 50)
            task_idx += 1

        print("#"*70)
        print("🔍 КОНЕЦ ТРАССИРОВКИ ОБРАТНОГО ПРОХОДА")
        print("#"*70 + "\n")
```

```c
// Быстрый генератор псевдослучайных чисел на основе индекса потока и seed
inline float nn_rand(int tid, unsigned int seed) {
    unsigned int x = tid + seed;
    x = (x ^ 61) ^ (x >> 16);
    x += (x << 3);
    x = x ^ (x >> 4);
    x *= 0x27d4eb2d;
    x = x ^ (x >> 15);
    // Возвращает float в диапазоне [0.0, 1.0]
    return (float)x / 4294967296.0f;
}

// Функция инлайн-дропаута для прямого прохода
inline float nn_dropout(float x, float p, int tid, unsigned int seed) {
    if (p <= 0.0f) return x;
    if (p >= 1.0f) return 0.0f;

    float r = nn_rand(tid, seed);
    if (r < p) {
        return 0.0f;
    } else {
        return x / (1.0f - p); // Масштабирование Inverted Dropout
    }
}

// Функция инлайн-дропаута для обратного прохода
inline float nn_dropout_backward(float grad_out, float p, int tid, unsigned int seed) {
    float r = nn_rand(tid, seed);
    if (r < p) {
        return 0.0f;
    } else {
        return grad_out / (1.0f - p);
    }
}
```

```python
    def __call__(self, lambda_func):
        try:
            raw_code = inspect.getsource(lambda_func).strip()
        except Exception:
            raise RuntimeError("LocalRunner не смог прочитать исходный код лямбды.")

        raw_code = " ".join(raw_code.split())
        match = re.search(r'(?:LocalRunner|runner)\(\s*lambda\s*:\s*(.*?)\s*\)\s*$', raw_code)
        if not match:
            match = re.search(r'lambda\s*:\s*([^)]*?)(?:\s*\)|$)', raw_code)

        expr_str = match.group(1).strip().replace('==', '=')
        expressions = [e.strip() for e in re.split(r'[;\n]', expr_str) if e.strip()]

        detected_tensors = {}
        saved_tensors_vram = {}  # Кэш клонированных буферов для текущей лямбды
        cl_body_forward = []
        cl_body_backward = []

        closure_vars = inspect.getclosurevars(lambda_func).nonlocals

        for expr in expressions:
            if '=' in expr:
                target, operation = expr.split('=', 1)
                target = target.strip()
                operation = operation.strip()

                # Подменяем python-вызовы nnclloc на Си-функции nn_
                cl_op = operation.replace('nncl1oc.', 'nn_')

                # ОЧИСТКА: Удаляем любые висячие закрывающие скобки
                while cl_op.count(')') > cl_op.count('('):
                    last_bracket = cl_op.rfind(')')
                    if last_bracket != -1:
                        cl_op = cl_op[:last_bracket] + cl_op[last_bracket+1:]

                # Находим все переменные (имена тензоров) внутри выражения
                tokens = re.findall(r'[a-zA-Z_][a-zA-Z0-9_]*', cl_op)
                for t in tokens:
                    # ЗАЩИТА: Исключаем Си-функции, типы данных, а также числовые переменные Python
                    if (t.startswith('nn_') or
                        t in ['float', 'int', 'tid', 'unsigned', 'p', 'seed', 'p_val', 'seed_val'] or
                        t in ['DROPOUT_P', 'DYNAMIC_SEED'] or # <-- ДОБАВИТЬ ИСКЛЮЧЕНИЕ ПЕРЕМЕННЫХ ИЗ ТЕСТА
                        t.isdigit()):
                        continue

                    detected_tensors[t] = True
```

```python
                tokens = re.findall(r'[a-zA-Z_][a-zA-Z0-9_]*', cl_op)
                for t in tokens:
                    # ЗАЩИТА: Исключаем Си-функции, типы данных, а также числовые переменные Python
                    if (t.startswith('nn_') or
                        t in ['float', 'int', 'tid', 'unsigned', 'p', 'seed', 'p_val', 'seed_val'] or
                        t in ['DROPOUT_P', 'DYNAMIC_SEED'] or # <-- ДОБАВИТЬ ИСКЛЮЧЕНИЕ ПЕРЕМЕННЫХ ИЗ ТЕСТА
                        t.isdigit()):
                        continue

                    detected_tensors[t] = True
                    # Индексируем только реальные плоские тензоры: имя -> имя[tid]
                    cl_op = re.sub(r'\b' + t + r'\b', f"{t}[tid]", cl_op)

                detected_tensors[target] = True
                cl_body_forward.append(f"{target}[tid] = {cl_op};")


                detected_tensors[target] = True
                cl_body_forward.append(f"{target}[tid] = {cl_op};")


                # 3. АВТО-BACKWARD (Полная изоляция через локальный upstream_grad)
                if self.is_training:
                    if "nn_dropout" in cl_op:
                        # Парсим аргументы: nn_dropout(выражение, p, tid, seed)
                        # Пример: nn_dropout(memory_state[tid] * gate_A[tid], 0.1f, tid, 54321)
                        drop_match = re.search(r'nn_dropout\((.*?), (.*?), tid, (.*?)\)', cl_op)
                        if drop_match:
                            inner_expr = drop_match.group(1) # то, что внутри дропаута (например, U * V)
                            p_val = drop_match.group(2)
                            seed_val = drop_match.group(3)

                            # Очищаем внутреннее выражение от индексов [tid] для разбора операндов в Python
                            clean_inner = inner_expr.replace('[tid]', '')

                            # Шаг А. Вычисляем градиент, просочившийся ЧЕРЕЗ маску дропаута.
                            # Мы вызываем зеркальную функцию nn_dropout_backward с ТЕМ ЖЕ tid и ТЕМ ЖЕ seed!
                            cl_body_backward.append(
                                f"float upstream_{target} = grad_{target}[tid];\n"
                                f"    float back_drop_{target} = nn_dropout_backward(upstream_{target}, {p_val}, tid, {seed_val});"
                            )

                            # Шаг Б. Передаем этот скорректированный градиент дальше по цепочке внутрь выражения.
                            # Если внутри дропаута было умножение (U * V):
                            if "*" in clean_inner:
                                u, v = [item.strip() for item in clean_inner.split('*')]
                                # Важно: используем back_drop_{target} вместо сырого grad_{target}
                                cl_body_backward.append(
                                    f"    grad_{u}[tid] += back_drop_{target} * {v}[tid];\n"
                                    f"    grad_{v}[tid] += back_drop_{target} * {u}[tid];"
                                )
                            elif "+" in clean_inner:
                                vars_add = [v.strip() for v in clean_inner.split('+')]
```

```
                    f"    float back_drop_{target} = nn_dropout_backward(upstream_{target}, {p_val}, tid, {seed_val});"
                )

                # Шаг Б. Передаем этот скорректированный градиент дальше по цепочке внутрь выражения.
                # Если внутри дропаута было умножение (U * V):
                if "*" in clean_inner:
                    u, v = [item.strip() for item in clean_inner.split('*')]
                    # Важно: используем back_drop_{target} вместо сырого grad_{target}
                    cl_body_backward.append(
                        f"    grad_{u}[tid] += back_drop_{target} * {v}[tid];\n"
                        f"    grad_{v}[tid] += back_drop_{target} * {u}[tid];"
                    )
                elif "+" in clean_inner:
                    vars_add = [v.strip() for v in clean_inner.split('+')]
                    for v in vars_add:
                        cl_body_backward.append(f"    grad_{v}[tid] += back_drop_{target};")
                else:
                    # Если внутри был просто одиночный тензор, градиент течет в него напрямую
                    cl_body_backward.append(f"    grad_{clean_inner}[tid] += back_drop_{target};")
            if "nn_sigmoid" in cl_op:
                src_var = re.search(r'nn_sigmoid\((.*?)\)', cl_op).group(1).replace('[tid]', '')
```

```
    # Сборка аргументов ядра
    kernel_hash = abs(hash(expr_str))
    kernel_name = f"fused_forward_{kernel_hash}"

    kernel_args = []
    for tensor in detected_tensors.keys():
        kernel_args.append(f"__global float* {tensor}")
        if self.is_training:
            kernel_args.append(f"__global float* grad_{tensor}")

    for saved_name in saved_tensors_vram.keys():
        kernel_args.append(f"__global const float* {saved_name}")

    args_str = ", ".join(kernel_args)
    forward_statements = "\n    ".join(cl_body_forward)

    full_forward_code = f"""
    {BASE_CL_FUNCTIONS}
    __kernel void {kernel_name}({args_str}, const int total_elements) {{
        int tid = get_global_id(0);
        if (tid >= total_elements) return;
        {forward_statements}
    }}
    """
    # >>> ВОТ СЮДА ВСТАВЛЯЕМ ПЕРВЫЙ БЛОК КОДА:
    # Находим реальные значения чисел из замыкания лямбды и вжигаем их в Си-текст
    if 'DROPOUT_P' in closure_vars:
        full_forward_code = full_forward_code.replace('DROPOUT_P', f"{float(closure_vars['DROPOUT_P'])}f")
    if 'DYNAMIC_SEED' in closure_vars:
        full_forward_code = full_forward_code.replace('DYNAMIC_SEED', f"{int(closure_vars['DYNAMIC_SEED'])}")
    program = JITProgramCache.get_or_compile(self.ctx, full_forward_code)
    kernel = getattr(program, kernel_name)

    first_tensor_name = list(detected_tensors.keys())[0]
    total_elements = closure_vars[first_tensor_name].size

    # --- ФИКСАЦИЯ БУФЕРОВ ДЛЯ FORWARD ВЫЗОВА ---
    cl_buffers_forward = []
    for tensor in detected_tensors.keys():
        tensor_obj = closure_vars[tensor]
        cl_buffers_forward.append(tensor_obj.data_buffer)
        if self.is_training:
            tensor_obj.require_grad()
            cl_buffers_forward.append(tensor_obj.grad_buffer)

    for vram_buffer in saved_tensors_vram.values():
        cl_buffers_forward.append(vram_buffer)

    # Выполняем прямой проход
    kernel(self.queue, (total_elements,), None, *cl_buffers_forward, np.int32(total_elements))
    self.queue.finish()
```

```
    # Выполняем прямой проход
    kernel(self.queue, (total_elements,), None, *cl_buffers_forward, np.int32(total_elements))
    self.queue.finish()

    # --- ИЗОЛИРОВАННАЯ ФИКСАЦИЯ БУФЕРОВ ДЛЯ BACKWARD ВЫЗОВА ---
    if self.is_training and cl_body_backward:
        backward_statements = "\n    ".join(cl_body_backward)
        backward_kernel_name = f"fused_backward_{kernel_hash}"

        full_backward_code = f"""
        {BASE_CL_FUNCTIONS}
        __kernel void {backward_kernel_name}({args_str}, const int total_elements) {{
            int tid = get_global_id(0);
            if (tid >= total_elements) return;
            {backward_statements}
        }}
        """
        # >>> ВОТ СЮДА ВСТАВЛЯЕМ ВТОРОЙ БЛОК КОДА:
        # Замораживаем маску для обратного прохода по тому же seed!
        if 'DROPOUT_P' in closure_vars:
            full_backward_code = full_backward_code.replace('DROPOUT_P', f"{float(closure_vars['DROPOUT_P'])}f")
        if 'DYNAMIC_SEED' in closure_vars:
            full_backward_code = full_backward_code.replace('DYNAMIC_SEED', f"{int(closure_vars['DYNAMIC_SEED'])}")
        # ВАЖНО: Мы сохраняем в стек КОПИЮ списка буферов (cl_buffers_forward[:]),
        # которая намертво зафиксировала состояние указателей и кэшей VRAM именно на ТЕКУЩИЙ шаг!
        cl_buffers_backward = list(cl_buffers_forward)

        self.backward_stack.append((full_backward_code, backward_kernel_name, cl_buffers_backward, total_elements))

def run_backward(self):
    if not self.backward_stack:
        print("-> Стек задач Backward пуст.")
        return

    print("\n" + "#"*70)
    print("🔍 ПОШАГОВАЯ ТРАССИРОВКА ОБРАТНОГО ПРОХОДА (DEEP DETAILED LOG)")
    print("#"*70)

    from compiler.host_returner import HostReturner
    import re

    task_idx = 1
    while self.backward_stack:
        code, name, buffers, total_elements = self.backward_stack.pop()

        print(f"\n[ЗАДАЧА НАЗАД №{task_idx}]: Вызов ядра {name}")
        print("-" * 50)
```

```
            print(f"\n[ЗАДАЧА НАЗАД №{task_idx}]: Вызов ядра {name}")
            print("-" * 50)

            # Извлекаем сигнатуру аргументов из сгенерированного Си-кода ядра
            # Нам нужно понять, в каком порядке идут указатели на буферы
            args_match = re.search(r'__kernel void \w+\((.*?)\)', code, re.DOTALL)
            if args_match:
                # Получаем список имён переменных из Си-кода, например: ['gate_A', 'grad_gate_A', 'saved_gate_A']
                arg_declarations = args_match.group(1).split(',')
                arg_names = [decl.strip().split()[-1].replace('*', '') for decl in arg_declarations if decl.strip()]
                # Последний аргумент обычно const int total_elements, отсекаем его
                if 'total_elements' in arg_names[-1]:
                    arg_names = arg_names[:-1]
            else:
                arg_names = [f"buffer_{i}" for i in range(len(buffers))]

            # Делаем дамп содержимого ВСЕХ задействованных буферов ДО выполнения текущего ядра
            print("Содержимое буферов во VRAM ПЕРЕД вычислением текущего шага:")
            with np.printoptions(precision=4, suppress=True, edgeitems=8, linewidth=80):
                for name_buf, cl_buf in zip(arg_names, buffers):
                    # Создаем временный суррогатный объект, чтобы HostReturner смог прочитать сырой cl.Buffer
                    class TargetWrapper:
                        def __init__(self, buf, size, queue):
                            self.data_buffer = buf
                            self.shape = (size,)
                            self.dtype = np.dtype(np.float32)
                            self.queue = queue

                    wrapper = TargetWrapper(cl_buf, total_elements, self.queue)
                    cpu_data = HostReturner.to_cpu(wrapper)
                    print(f"  • {name_buf:18} -> {cpu_data}")

            # Компилируем и запускаем ядро на GPU
            program = JITProgramCache.get_or_compile(self.ctx, code)
            kernel = getattr(program, name)
            kernel(self.queue, (total_elements,), None, *buffers, np.int32(total_elements))
            self.queue.finish() # Синхронизируем, чтобы увидеть изменения

            # Делаем дамп результатов ПОСЛЕ выполнения ядра
            print("\nСодержимое буферов во VRAM ПОСЛЕ вычисления текущего шага:")
            with np.printoptions(precision=4, suppress=True, edgeitems=8, linewidth=80):
                for name_buf, cl_buf in zip(arg_names, buffers):
                    wrapper = TargetWrapper(cl_buf, total_elements, self.queue)
                    cpu_data = HostReturner.to_cpu(wrapper)
                    print(f"  • {name_buf:18} -> {cpu_data}")

            print("-" * 50)
            task_idx += 1

        print("#"*70)
        print("🔍 КОНЕЦ ТРАССИРОВКИ ОБРАТНОГО ПРОХОДА")
        print("#"*70 + "\n")

```